Agentic Coding Workflow Guideline

Evidence-Based Autonomy, Good-Enough Passes, and Behavior-Preserving Code Transformation

Core principle

Let the LLM decide scope, approach, and what is good enough for the current pass. The system must still enforce stage boundaries, schema discipline, deterministic validation, rollback, and traceability.

This document is intended to guide an agentic coding repository that organizes, cleans, documents, tests, debugs, and eventually translates code while preserving working behavior. It captures the workflow pattern proven during testing: Planner defines good enough, Test Designer defines evidence, Applier does only enough work, Validator proves the repo still works, and Final Reviewer judges whether the outcome and evidence match.

Table of Contents

1. Mission and Scope

2. Main Pillars: What the Agent Must Do and Must Not Do

3. Evidence-Based Autonomy

4. The Good-Enough Pass Pattern

5. Workflow Modes

6. Agent Roles and Responsibilities

7. Stage Boundaries

8. Behavior Preservation and No-Touch Rules

9. Verify-First Bug Fixing

10. Validation, Evidence, and Test Quality

11. Schema Discipline and Defensive Normalization

12. Safe Failure, Rollback, and Traceability

13. Environment-Aware Tooling

14. Model Routing and Reliability

15. Translation and Larger Restructuring

16. Evaluation Checklist

17. Operating Procedure

18. Prompt Design Guidelines

19. Lessons from the Test Runs

20. Reference-Inspired Design Notes

21. Final Guideline Summary

References and Research Inspirations

22. Research-Based Refinements Added After Workflow Testing

23. Updated Final Operating Model

24. Additional Research References Added

1. Mission and Scope

The mission of the agentic coding workflow is to help a developer or researcher organize, clean, document, test, debug, and eventually translate repositories while preserving working behavior. The system is designed for practical research code: messy but valuable repositories where the working algorithmic logic may matter more than perfect style.

The workflow should support bounded progression: from small single-file edits, to coordinated multi-file changes, to optional larger restructuring once dependency understanding, validation, interface preservation, and traceability are strong enough.

- Primary near-term goal: organize, clean, document, and test code while preserving behavior.
- Secondary goal: make the workflow general across cleanup, documentation, tests, bug fixes, and future translation modes.
- Long-term goal: support larger repository restructuring only when evidence, validation, and traceability are mature enough.
- Non-goal: letting the LLM freely rewrite working algorithms without proof, review, or explicit authorization.

Mission statement

Create a workflow where the LLM can make engineering judgments, but every judgment is grounded in repo context, guidelines, deterministic validation, and reviewable evidence.

2. Main Pillars: What the Agent Must Do and Must Not Do

These main pillars are the practical operating rules for the agentic coding workflow. They translate the overall philosophy into explicit do and do-not behaviors. The highest-level pillar is: Autonomy with evidence. Flexibility with validation. Creativity with rollback.

> *Highest-level guiding principle: Give the LLM autonomy over engineering judgment, but require evidence, validation, stage discipline, and rollback protection before accepting its work.*

2.1 Evidence-Based Autonomy

The LLM is allowed to decide scope, approach, and what is good enough, but every autonomous decision must be tied to evidence.

Do

- Use repo context, file contents, guidelines, task instructions, validation output, and reviewer judgment to make decisions.
- Define what good enough means for this pass and explain why the pass is sufficient.
- List what is intentionally deferred so the agent does not chase perfection.
- Use validation evidence to support the final outcome.

Do not

- Do not claim success without validation.
- Do not invent unsupported facts about the repo.
- Do not make changes merely because a task sounds like it requires changes.
- Do not assume a bug exists without checking.
- Do not treat LLM confidence as proof.

> *Core principle: The LLM may decide, but evidence must justify the decision.*

2.2 Preserve Working Behavior

The default assumption is that working behavior should remain unchanged unless the task explicitly requires behavior change.

Do

- Preserve public APIs, function behavior, file outputs, CLI behavior, and test-visible behavior.
- Use existing tests and known outputs as behavioral references.
- Add tests before or during risky changes.
- Prefer small, coherent changes when behavior preservation matters.
- Document intentional behavior changes clearly and only make them when authorized by the task.

Do not

- Do not rewrite working algorithmic logic just to make it cleaner.
- Do not delete compatibility wrappers unless explicitly approved.
- Do not change outputs, filenames, formats, or numeric behavior casually.
- Do not simplify code in a way that changes edge-case behavior.
- Do not refactor validated scientific or algorithmic code without authorization.

> *Core principle: Clean the structure around working logic before touching the working logic itself.*

2.3 Good-Enough Pass Design

Each run should be a bounded engineering pass, not an attempt to solve everything at once.

Do

- Define the intended outcome for the pass.
- Define what good enough means for this specific pass.
- Define what evidence proves the pass is good enough.
- Do only enough work to satisfy the plan and mode.
- Defer nonessential work explicitly.

Do not

- Do not keep expanding scope mid-run.
- Do not turn a small cleanup into a full repo rewrite.
- Do not add unnecessary architecture.
- Do not chase perfection.
- Do not hide deferred work.

> *Core principle: A pass is successful when it achieves the defined outcome with evidence, not when it does everything possible.*

2.4 Verify Before Changing

For bug fixes and uncertain tasks, the first step is verification.

Do

- Check whether the bug still exists.
- Inspect the relevant source and tests.
- Run or propose a direct reproduction command.
- If the bug exists, apply the smallest coherent fix.
- If the bug is already fixed, preserve the code and verify coverage.

Do not

- Do not invent a fix for a bug that is already fixed.
- Do not edit source code when validation shows no source change is needed.
- Do not rely on assumptions about import behavior, paths, state, or outputs.
- Do not treat the task wording as proof that the bug exists.

> *Core principle: First verify reality; then decide whether a change is needed.*

2.5 Tests Are Evidence, Not Decoration

Tests should prove that the pass achieved its intended behavior-preserving or behavior-fixing goal.

Do

- Design tests around behavior that must be preserved, fixed, or translated.
- Use regression tests for bug fixes.
- Use smoke tests for end-to-end workflows.
- Use reference-output comparison for translation modes.
- Run existing tests after changes.

Do not

- Do not add tests that merely confirm the new implementation does something.
- Do not create fragile tests based on timestamps or environment-specific paths unless normalized.
- Do not add heavyweight test infrastructure unless needed.
- Do not claim test coverage is useful without explaining what behavior it proves.

Core principle: A test matters only if it proves the behavior the pass claims to preserve or improve.

2.6 Deterministic Validation Is the Source of Truth

The Validator proves whether the repo still works after the LLM proposes and applies a change.

Do

- Run compile checks, import checks, unit tests, smoke tests, and mode-specific validation.
- Capture command, working directory, stdout, stderr, and exit code.
- Use validation results in final review.
- Treat missing validation evidence as incomplete work.

Do not

- Do not accept a change only because the LLM says it is correct.
- Do not skip validation after source changes.
- Do not ignore failed validation.
- Do not call a run successful when validation did not execute.

Core principle: The LLM proposes; validation proves.

2.7 Stage Boundaries Must Be Clear

Each agent should judge only the stage it is responsible for.

Do

- Use Plan Reviewer to judge whether a plan is worth attempting.
- Use Final Reviewer to judge completed work after validation.
- Keep Planner, Test Designer, Applier, Validator, Repair, and Reviewer roles distinct.
- Make each stage produce the evidence needed by the next stage.

Do not

- Do not make the Plan Reviewer require validation results before work has been applied.
- Do not make the Applier redesign the entire plan.
- Do not let the Final Reviewer ignore validation.
- Do not collapse all decisions into one vague review stage.

Core principle: Plan review asks, "Is this worth trying?" Final review asks, "Did this actually work?"

2.8 Schema Discipline With Defensive Normalization

LLM outputs must be structured enough for the system to use safely, but the system should recover common near-correct outputs when safe.

Do

- Require JSON schemas for each agent.

- Normalize safe near-misses, such as moving existing files from added_files to changed_files.
- Fill missing reasons with conservative defaults when the intent is unambiguous.
- Convert top-level intended_change into planned_changes when clearly recoverable.
- Reject unrecoverable malformed outputs and log normalization decisions.

Do not

- Do not trust arbitrary prose as a structured result.
- Do not silently accept malformed plans.
- Do not normalize changes that are semantically ambiguous.
- Do not allow schema failure to corrupt the repo.

> *Core principle: The LLM can be flexible in reasoning, but the system needs structured outputs to act safely.*

2.9 Safe Failure and Rollback Are Success Behaviors

A run can be successful from a safety perspective even when the candidate is not accepted.

Do

- Stage candidate changes before applying.
- Back up original files.
- Run validation after applying.
- Revert if validation or final review fails.
- Report why the pass was not good enough and preserve candidate artifacts when useful.

Do not

- Do not leave the repo partially modified after failed validation.
- Do not treat a reverted run as useless.
- Do not hide failure details.
- Do not continue applying more changes after a decisive validation failure.

> *Core principle: A safe revert is better than an unsafe success claim.*

2.10 Environment-Aware Execution

The agent must respect the actual machine, shell, and toolchain it is running on.

Do

- Consider OS, shell, path style, working directory, installed tools, and available compilers.
- Use Windows-compatible commands on Windows and shell-appropriate syntax everywhere.
- Check whether tools like cmake, g++, cl, pytest, or git exist before relying on them.
- Prefer Python comparison scripts over Unix-only tools when cross-platform behavior matters.

Do not

- Do not assume Linux commands like diff, ./executable, or bash syntax on Windows.
- Do not assume CMake or a compiler exists.
- Do not assume executable paths without checking the build system.
- Do not call an environment limitation a code failure without explaining the distinction.

Core principle: Validation commands must be executable in the actual environment, not just reasonable in theory.

2.11 Reference Implementation for Translation

For translation modes, the original implementation is the behavioral reference.

Do

- Keep the original implementation unchanged.
- Generate translated code separately.
- Compare translated behavior against the reference using the same input data.
- Compare semantic outputs, not fragile timestamps or path strings.
- Use tolerances for floating-point differences.

Do not

- Do not overwrite the source implementation.
- Do not judge translation quality only by whether the code compiles.
- Do not require byte-for-byte equality when timestamps, path separators, float formatting, or unordered map order differ.
- Do not copy input data unnecessarily if the translated code can reference the existing data.

Core principle: Translation is good enough only when the generated implementation matches reference behavior.

2.12 Minimal Necessary Change

The Applier should do the smallest amount of work needed to satisfy the approved plan.

Do

- Touch only the files needed for the pass.
- Keep changes coherent and local when possible.
- Prefer local edits over broad rewrites.
- Document why each file changed.
- Leave deferred work for later passes.

Do not

- Do not modify unrelated files.
- Do not perform opportunistic cleanup outside the task.
- Do not restructure the repo unless the mode and plan justify it.
- Do not add new dependencies unless necessary.

Core principle: Do enough to satisfy the pass, not everything the model can think of.

2.13 Human-Readable Traceability

Every run should leave behind a clear explanation that future engineers can understand.

Do

- Record task, mode, plan, test plan, apply result, validation result, final review, changed files, and next steps.
- Explain why the pass was good enough or not good enough.

- Include validation commands and outputs.
- List deferred work.

Do not

- Do not produce vague summaries.
- Do not hide uncertainty.
- Do not omit failed commands.
- Do not claim behavior preservation without evidence.

> *Core principle: A future engineer should be able to understand what happened, why it happened, and how it was validated.*

2.14 Model Strength Helps, but Workflow Reliability Matters More

Better models reduce errors, but the system must not rely on perfect model behavior.

Do

- Use stronger models for reviewer, planner, or high-risk modes when needed.
- Keep model routing configurable.
- Design the pipeline to tolerate imperfect outputs.
- Use validation and repair loops regardless of model strength.

Do not

- Do not assume upgrading the model removes the need for schema checks.
- Do not remove validation because the model is stronger.
- Do not trust a confident model over failing tests.

> *Core principle: Better models help, but reliable workflow design is the foundation.*

3. Evidence-Based Autonomy

Evidence-Based Autonomy is the central philosophy. The LLM is allowed to decide the scope, approach, and sufficiency of a pass. However, those autonomous decisions must be tied to explicit evidence: repository context, task instructions, guideline rules, validation commands, test results, and final review.

- Autonomy without evidence becomes hallucination risk.
- Evidence without autonomy becomes rigid automation that cannot adapt to real repositories.
- The correct balance is to let the LLM decide what is enough, then require it to explain and prove that decision.

Autonomous decision	Required evidence
Which files to touch	Repo map, file context, dependency relevance, and plan rationale
Whether a bug exists	Direct inspection, targeted import/runtime checks, or failing test evidence
Whether no source change is needed	Passing targeted validation and regression coverage
Whether a pass is good enough	Completion criteria, validation result, and final reviewer approval
Whether to defer work	Clear explanation of why it is out of scope for this pass

4. The Good-Enough Pass Pattern

The “good enough” pattern is the key mechanism that makes the workflow flexible without becoming uncontrolled. It does not impose arbitrary file-count or edit-count restrictions. Instead, it asks the agents to define the desired outcome, the evidence needed to prove it, and the boundary of the current pass.

Stage	Good-enough responsibility
Planner	Defines the intended outcome, scope, target files, risks, deferred work, and why this pass would be good enough.
Plan Reviewer	Checks whether the plan is coherent and worth attempting. It must not demand validation evidence yet.
Test Designer	Defines what behavior must be preserved and what evidence will prove the pass is good enough.
Applier	Performs only the work needed to satisfy the plan and mode. It should not broaden scope.
Validator	Runs deterministic checks and records objective pass/fail evidence.
Repair Agent	Fixes only validation failures caused by the candidate. It should not introduce new goals.
Final Reviewer	Judges whether the completed outcome, validation evidence, and good-enough explanation match.

Good-enough rule

A pass is good enough when the intended outcome is satisfied, behavior is preserved or intentionally fixed, validation evidence supports the claim, and deferred work is clearly named rather than hidden.

5. Workflow Modes

Workflow modes make the repository general. Instead of hardcoding one behavior, the same engine can operate in cleanup, document-only, test-generation, bug-fix, or future translation modes. Each mode should define purpose, output policy, validation profile, file scope, and special notes.

Mode	Purpose	Good-enough emphasis
cleanup	Organize, clean, document, and test code while preserving behavior.	Improved readability/organization with validation proving behavior still works.
document_only	Improve README, docstrings, comments, and architecture notes.	Documentation improved without source logic changes.
test_generation	Add or improve unit/smoke/regression tests.	Tests increase useful evidence without unnecessary source changes.
bug_fix	Fix or verify a specific bug.	Bug state verified first; smallest fix

		if needed; no invented fix if already resolved.
translate_cpp	Create a C++ implementation while preserving the Python reference.	Candidate isolated under cpp/, Python preserved, C++ build/run evidence required.

- Modes should be passed to every agent prompt.
- Modes should not replace judgment; they guide it.
- A mode should describe what success means and what types of changes are allowed.
- Adding a new mode should require minimal code changes: update modes.py, optionally validation profiles, and mode-aware prompt wording.

6. Agent Roles and Responsibilities

Each agent should have a distinct responsibility. When responsibilities blur, the workflow becomes unstable. The system should not ask the Planner to prove validation, nor the Reviewer to apply fixes, nor the Applier to invent a new task.

Agent	Should do	Should not do
Mapper	Summarize repo structure, package roots, entrypoints, tests, risks, and related files.	Propose edits or rewrite code.
Planner	Define target outcome, target files, planned changes, risks, validation strategy, and good-enough reason.	Return command lists, apply patches, or require validation evidence before work is attempted.
Plan Reviewer	Approve/revise/reject whether plan is worth attempting.	Reject a plan just because apply_result or validation_result are absent.
Test Designer	Define behavior to preserve and commands/tests that produce evidence.	Create heavyweight testing infrastructure unless justified.
Applier	Return full file contents for changed/added files and stay within planned scope.	Broaden scope, omit reasons, or invent fixes when no source change is needed.
Validator	Run deterministic commands and capture stdout/stderr/exit codes.	Make subjective claims beyond command results.
Repair Agent	Fix only validation failures caused by the candidate.	Start a new refactor or broaden task scope.
Final Reviewer	Judge final outcome using apply result and validation evidence.	Judge a plan as if it were completed work.

7. Stage Boundaries

Stage boundaries are one of the most important practical design lessons. A plan-review prompt must be different from a final-review prompt. Otherwise the reviewer may reject a plan because validation has not run yet, which is expected at that stage.

Stage	Available evidence	Reviewer question
Plan review	Task, guideline, repo map, file context, and plan.	Is this plan coherent and safe enough to attempt?
Test design	Approved plan plus file context.	What evidence would prove the plan succeeded?
Apply	Plan, test plan, file contents, and mode.	What files need to be changed or added?
Validation	Applied candidate plus validation commands.	Did objective checks pass?
Final review	Plan, apply result, validation result, and mode.	Is the completed result actually good enough?

Permanent rule

Plan review is not final review. Plan reviewers should evaluate coherence and safety. Final reviewers should evaluate completed work and validation evidence.

8. Behavior Preservation and No-Touch Rules

The workflow is designed for repositories where working code may be more valuable than clean code. A cleanup pass should improve organization and understanding without changing validated behavior. Algorithmic or scientific logic deserves extra protection.

- Never change code that already works unless the task explicitly requires a behavior change.
- Validated algorithms are protected. Cleanup should happen around them unless algorithm modification is explicitly authorized.
- Preserve public APIs, entrypoints, file outputs, CLI behavior, and test-visible behavior unless the task says otherwise.
- If a source change is unnecessary, the correct action may be no source change plus validation evidence.
- Outcome matters more than edit path, but working behavior must remain intact.

Allowed by default	Requires explicit authorization
Docstrings, README improvements, comments, architecture notes	Changing algorithmic logic
Small refactors that preserve behavior and pass tests	Deleting public functions or compatibility aliases
Test additions that capture current behavior	Changing expected behavior to match a mistaken test
Module organization improvements with validation	Large repo restructuring without dependency understanding

9. Verify-First Bug Fixing

Bug-fix mode must start with verification. The agent should not assume the bug exists merely because the task says “fix.” The correct fix may be to prove the bug is already fixed and preserve regression coverage.

1. Inspect the relevant files and repo context.

2. Run or propose a targeted validation command that directly checks the alleged bug.
3. If the bug exists, apply the smallest coherent behavior-preserving fix.
4. If the bug is already fixed, do not invent a source-code change.
5. Add or preserve a regression test that proves the behavior.
6. Run deterministic validation and record the evidence.

Example principle

When an import already works, the correct result can be: no source changes, existing regression test preserved, targeted import command passed, full test suite passed.

10. Validation, Evidence, and Test Quality

Validation is the source of objective evidence. The LLM may decide what is good enough, but validation proves whether the repo still works. Tests are evidence only when they directly support the behavior claimed by the pass.

- Always run syntax checks for languages supported by the mode.
- Run existing tests before trusting new behavior.
- Run targeted commands for the specific behavior being changed or verified.
- Use smoke tests to prove entrypoints still run.
- For translations, compare outputs semantically when timestamps or floating-point formatting prevent byte-for-byte comparison.
- Do not treat a test as valuable simply because it exists; it must prove the intended behavior.

Evidence type	What it proves	Limitations
compileall / syntax check	Files parse and basic syntax is valid.	Does not prove logic is correct.
Unit tests	Specific functions behave as expected.	Can encode wrong assumptions if not reviewed.
Smoke tests	Entrypoints run end-to-end.	May miss edge cases.
Targeted bug command	Specific alleged bug is fixed or absent.	Must be chosen carefully.
Final reviewer judgment	Outcome and evidence match the task.	Depends on prompt clarity and model reliability.

11. Schema Discipline and Defensive Normalization

LLMs often return nearly correct JSON: missing a field, placing an existing file under added_files, or using a task-list format. The system should enforce schemas, but it should also normalize common safe mistakes. This is not over-restricting the LLM; it is making the workflow robust.

- Planner must return one CleanupPlan object, not a task list or command list.
- Applier must return one ApplyResult object with summary, changed_files, added_files, deleted_files, and notes.
- Every file edit must include path, content, and reason.

- Existing files should be placed in changed_files, not added_files.
- Repair must return a repair result, not free-form analysis.

Common LLM mistake	Safe normalization
Top-level intended_change but missing planned_changes	Wrap it into planned_changes using target_files[0] when clearly recoverable.
Existing file appears in added_files	Move it to changed_files when file_context proves it already exists.
Missing reason field	Fill conservative default reason and continue.
JSON list instead of object	Reject unless it is a recoverable list of file edits.
Analysis before JSON	Improve prompt and JSON extraction; reject unrecoverable outputs.

12. Safe Failure, Rollback, and Traceability

A failed run is not necessarily a system failure. If the candidate is isolated, validation catches the issue, original files are restored, and the report explains why the result was not good enough, then the system behaved safely.

- Use a workspace/candidate area before promoting changes.
- Back up originals before applying candidate files.
- Revert when validation fails or final review rejects the result.
- Record the run ID, mode, task, plan, apply result, validation result, final status, changed files, and next recommended steps.
- Treat safe failure as useful feedback, especially in new modes such as translation.

Safe failure definition

A reverted run is acceptable when it preserves the original repo, exposes concrete blockers, and produces actionable next steps.

13. Environment-Aware Tooling

Coding agents must know the execution environment. A command that is valid on Linux may fail on Windows. Translation and build workflows are especially sensitive to available tools such as CMake, compilers, shell syntax, and path conventions.

Environment factor	Guideline
Operating system	Generate commands appropriate for Windows, Linux, or macOS.
Shell	Avoid bash syntax in cmd/PowerShell unless explicitly available.
Build tools	Check whether cmake, compiler, pytest, git, or other tools exist before requiring them.
Path style	Use paths compatible with the active environment.
Executable invocation	Use .exe or correct build output path on Windows; avoid ./ when using cmd.
Comparison tools	Use Python comparison scripts instead of assuming

	diff/fc availability.

- Validation commands must be executable in the current environment.
- When tools are missing, the result should be a safe failure with clear setup instructions or fallback commands.
- For C++ translation, install CMake or provide manual compiler fallback before expecting build validation to pass.

14. Model Routing and Reliability

Different model roles benefit from different capabilities. A mapper may need broad summarization; a planner needs structured reasoning; an applier needs code generation; a reviewer needs critical judgment. The system should support model routing while remaining robust to transient cloud errors and malformed outputs.

Role	Model quality needed	Reliability guardrail
Mapper	Repository summarization and symbol awareness.	Limit context and avoid edit proposals.
Planner	Reasoning, scope selection, good-enough definition.	Strict schema plus plan normalization.
Reviewer	Critical judgment and technical correctness.	Separate plan-review and final-review prompts.
Applier	Code generation and file-content consistency.	Schema normalization and validation.
Repair	Failure diagnosis and minimal fix.	Repair-only prompt and limited context.

- A stronger model can reduce mistakes but cannot replace validation and rollback.
- Add retry logic for transient 429/502/503/504 cloud/model failures.
- Do not trust a model because it sounds confident; trust the workflow when evidence supports the result.

15. Translation and Larger Restructuring

Translation and repo-wide restructuring should be treated as advanced modes. They require stronger validation, environment awareness, and staged expectations. A translation pass can safely create a candidate under a separate directory while preserving the original source.

- Keep the source implementation as reference.
- Create translated code under a separate directory such as cpp/ rather than overwriting source files.
- Build validation must be available before the pass can be considered good enough.
- Semantic output comparison may be better than byte-for-byte comparison when timestamps or floating-point formatting differ.
- Large restructuring should only occur after dependency understanding, interface preservation, scientific correctness checks, and traceability are strong enough.

Advanced workflow	Required readiness
Python to C++	CMake/compiler available, Windows-aware commands, output comparison strategy.

Python to MATLAB	MATLAB execution or clear manual validation strategy.
Repo-wide restructuring	Dependency map, tests, interface inventory, rollback, staged apply.
Scientific algorithm refactor	Explicit authorization, reference outputs, numerical tolerance, and domain review.

16. Evaluation Checklist

Use this checklist after each run to determine whether the agentic workflow is improving and whether the result can be trusted.

- Did the selected workflow mode match the task?
- Did the Planner define a clear intended outcome?
- Did the Planner explain what good enough means?
- Did the Plan Reviewer judge the plan at the correct stage?
- Did the Test Designer define useful evidence rather than generic tests?
- Did the Applier stay within scope?
- Were existing working behaviors preserved?
- Did deterministic validation pass?
- Did the Final Reviewer compare outcome, evidence, and good-enough claim?
- If the run failed, did it fail safely and revert?
- Were next steps actionable?

17. Operating Procedure

The following procedure should be used when operating the repo cleaner.

7. Start with modes and models commands to confirm configuration.
8. Run map to confirm the target repo is readable.
9. Run validate to establish baseline health.
10. Run plan for the selected mode before apply when the mode is new or risky.
11. Run apply only when the plan is coherent.
12. Inspect final_status, changed_files, validation_result, and good_enough_reason.
13. Treat safe failures as useful feedback, not as reasons to weaken validation.
14. Update guidelines/prompts only when failures reveal a reusable pattern.
15. Prefer strengthening stage boundaries and normalization before adding arbitrary constraints.

18. Prompt Design Guidelines

Prompts should be detailed enough to define roles and schema, but they should not overconstrain engineering judgment. They should describe goals, boundaries, and output contracts.

Prompt design rule	Reason
Every agent prompt should state the workflow mode.	Agents must know the task type and output policy.
Every schema prompt should forbid arrays/task lists	Prevents common malformed outputs.

when objects are required.	
Reviewer prompts must be stage-specific.	Prevents plan review from requiring completed-work evidence.
Bug-fix prompts must say verify first.	Prevents invented fixes.
Applier prompts must require full file contents and reasons.	Makes apply result auditable.
Repair prompts must forbid broadening scope.	Keeps repair focused on validation failures.

Prompt length principle

A long prompt is not automatically better. A useful prompt defines the role, stage, mode, schema, allowed judgment, forbidden mistakes, and evidence expectations.

19. Lessons from the Test Runs

The test sequence showed that the generalized workflow works for Python cleanup, documentation, test generation, and verify-first bug fixing. It also showed that translation modes require environment support and stricter build validation.

Test area	Lesson
cleanup	The good-enough pattern supports practical cleanup without arbitrary constraints.
document_only	Mode-specific prompts can keep changes documentation-only.
test_generation	The agent can improve tests while preserving source behavior.
bug_fix	Verify-first wording and bug_fix mode notes prevented invented source changes.
schema robustness	Planner and Applier normalization are necessary for near-correct model outputs.
reviewer stages	Plan review and final review must use different prompts.
translate_cpp	Safe failure worked; C++ validation needs CMake/compiler and Windows-aware commands.

20. Reference-Inspired Design Notes

The workflow aligns with several themes from recent agentic coding and software-engineering-agent work. SWE-agent emphasizes that the agent-computer interface matters: tools for editing, navigation, tests, and execution influence agent effectiveness. SWE-bench frames real software engineering as solving issues in real repositories using codebase context and patches. Recent work on agent-generated tests reinforces that tests are useful only when they prove the intended behavior, not merely because they were generated. These ideas support the design direction: scoped autonomy, environment-aware tool use, deterministic validation, and evidence-based review.

- Agent-computer interface matters: make repository navigation, file editing, commands, and validation clear.
- Real software tasks require context, not just isolated snippets.
- Validation and tests are central to trust.
- Generated tests must be evaluated for behavioral relevance.
- Human and LLM roles should shift toward orchestration, verification, and evidence review rather than blind code generation.

21. Final Guideline Summary

- Let the LLM decide what is good enough, but require it to define that decision clearly.
- Use deterministic validation as objective evidence.
- Keep plan review separate from final review.
- Protect working behavior, especially validated algorithms and research logic.
- Verify bugs before fixing them; do not invent source-code changes.
- Normalize common schema mistakes, but reject unsafe or unrecoverable outputs.
- Treat rollback as a success mode when validation catches bad candidates.
- Make validation environment-aware.
- Keep multi-agent complexity optional until the single orchestrated workflow is reliable.
- Use the final reviewer to judge whether outcome, explanation, and evidence match.

North-star pattern

Planner defines good enough. Test Designer defines evidence. Applier does only enough work. Validator proves. Final Reviewer judges. The developer remains in control of goals, guidelines, and promotion decisions.

References and Research Inspirations

- **SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering** — NeurIPS 2024. Emphasizes that agent-computer interface design improves agent ability to edit code, navigate repositories, and execute tests.
- **SWE-bench: Can Language Models Resolve Real-World GitHub Issues?** — ICLR 2024 / arXiv 2023. Frames software-engineering evaluation around real GitHub issues, codebase context, and patches.
- **Rethinking the Value of Agent-Generated Tests for LLM-based Coding Agents** — arXiv 2026. Supports the guideline that tests must be evaluated for behavioral relevance, not just generated.
- **Surveys of LLM-based agents and scientific agents** — Recent surveys emphasize autonomy, tool use, research lifecycle integration, verification, and reliability as central challenges.

Note: This guideline is written as an operational design document. The references are included as inspiration for design principles, not as hard dependencies for the repository implementation.

22. Research-Based Refinements Added After Workflow Testing

This section adds refinements drawn from recent agentic software engineering publications and from the observed test behavior of this repository. The core conclusion is that agentic coding systems should not rely on model intelligence alone. They work best when model autonomy is paired with a strong agent-computer interface, stage-specific prompts, deterministic validation, environment awareness, rollback, and evidence-based review.

21.1 Agent-Computer Interface as a First-Class Design Concern

Recent work such as SWE-agent emphasizes that language-model coding agents are a new class of computer users. Their success depends not only on the base model, but also on the interface they receive for repository navigation, file editing, command execution, and test feedback. This guideline therefore treats the agent-computer interface as part of the system architecture, not as an implementation detail.

- The agent should receive a structured repo map, not just raw file text.
- The agent should receive relevant file contents and clear file paths for the current pass.
- The agent should know the working directory, operating system, shell assumptions, and available tools.
- The agent should receive validation output in a form that is easy to reason over: command, cwd, exit code, stdout, stderr, and failure summary.
- The agent should be told whether it is reviewing a plan, applying code, repairing validation failures, or judging completed work.

Design rule: improve the agent interface before adding agent complexity. A weak interface can make a strong model behave poorly; a strong interface can make a modest model more reliable.

21.2 Environment-Aware Validation

The C++ translation test demonstrated an important system lesson: a validation plan can be logically correct but environmentally invalid. Commands such as diff, ./build/program, or cmake may fail on Windows if the tools are not installed or the shell syntax is wrong. The guideline should require environment-aware validation before a workflow is judged as failed.

- Validation commands must match the operating system and shell.
- The agent must not assume Unix commands such as diff, chmod, ./program, or rm on Windows.
- The agent must not assume CMake, gcc, g++, cl, make, pytest, or other tools exist unless the environment confirms them.
- When a tool is missing, the report should distinguish environment limitation from code failure.
- For cross-platform comparison, prefer Python comparison scripts over shell-specific commands when practical.
- For Windows C++ validation, prefer commands such as where cmake, cmake --version, where cl, where g++, and Python-based output comparison.

Good-enough implication: a translation pass is not good enough merely because files were generated. It becomes good enough only when the environment can build or otherwise credibly validate the generated implementation.

21.3 Generate-and-Validate Loop

Agentic software engineering literature increasingly frames coding as an iterative generate-and-validate process. This aligns directly with the repository pattern tested here: the LLM may decide the scope and approach, but validation decides whether the result is acceptable.

- For cleanup: compile, import, run tests, run smoke checks.
- For documentation: confirm syntax/tests still pass and documentation changes do not alter runtime behavior.
- For test generation: confirm new tests pass and actually exercise intended behavior.
- For bug fixing: reproduce or verify the bug first; then fix or confirm no fix is needed; then run regression validation.
- For translation: run the reference implementation, build the translated implementation, run it, and compare semantic output.

The validator is not a formality. It is the objective evidence layer that prevents the workflow from becoming unconstrained vibe coding.

21.4 Stage-Specific Review

One of the most important findings from testing was that plan review and final review must be different prompts. A plan reviewer should not reject a plan because apply_result or validation_result is empty; at that stage, no files have been changed yet. A final reviewer, however, should require actual validation evidence.

16. Plan Reviewer: Is this plan coherent enough to attempt? Are target files reasonable? Is the validation strategy appropriate? Is good enough defined?
17. Final Reviewer: Did the applied work match the plan? Did validation pass? Does the evidence prove the good-enough claim? Should the candidate be accepted, repaired, or reverted?

Hard rule: every reviewer prompt must declare which stage is being reviewed. Ambiguous reviewer prompts create false failures.

21.5 Schema Robustness and Output Normalization

The tested system showed that models often produce almost-correct structured outputs: a plan missing planned_changes, an existing file placed under added_files, or a file edit missing reason. The system should enforce schemas, but it should also normalize safe, obvious mistakes.

- If an existing file appears in added_files, move it to changed_files.
- If a file edit is missing reason, add a conservative default reason.
- If the Planner returns intended_change and behavior_preservation_notes but omits planned_changes, wrap them into a single planned change.
- If the model returns a JSON array or task list instead of the required object, reject or retry rather than guessing.
- Every normalization should be traceable so the final report can distinguish model output from system correction.

This is not over-restricting the LLM. It is the reliability layer that allows the LLM to remain flexible without making the pipeline brittle.

21.6 Tests as Evidence, Not Decoration

A recurring theme in agentic coding is that tests can create false confidence if they merely assert whatever the generated implementation already does. In this workflow, tests must be treated as evidence for the intended behavior, not as decorative artifacts.

- A useful test checks behavior that the guideline, existing implementation, or reference implementation says must hold.
- A weak test only checks that the current generated code returns something.
- The Test Designer must explain what behavior each proposed test proves.
- The Final Reviewer must judge whether the tests actually support the good-enough claim.
- When behavior is already fixed, preserving a passing regression test may be better than inventing a source-code change.

21.7 Safe Failure and Rollback as a Success Condition

The C++ translation run failed validation and was reverted. That is a good system outcome. A failed candidate is not a failed workflow if the repo is protected, the failure is explained, and the next step is clear.

- Original files must be restored when validation or final review fails.
- Candidate files and validation failures should be recorded for debugging.
- The report should explain whether failure was due to code quality, missing tools, platform mismatch, bad commands, or insufficient evidence.
- A reverted run should be considered a safe failure when it prevents unvalidated code from landing.

Guideline principle: safe failure is part of trust. A system that refuses to apply questionable work is more useful than a system that always forces a change.

21.8 Translation and Reference Implementation Rules

For translation modes, the original implementation is the reference. A translation is not judged by whether it looks reasonable; it is judged by whether it preserves behavior against the reference implementation.

- Run the original implementation on known inputs before judging the translation.
- Run the translated implementation on the same inputs when tools are available.
- Compare semantic output: totals, groups, row counts, key structures, and error behavior.
- Avoid byte-for-byte comparison when timestamps, path separators, unordered map ordering, or floating-point string formatting may differ.

- Use tolerance-based numeric comparison and structure-aware JSON comparison when exact text comparison is inappropriate.
- Do not duplicate input data unless necessary; prefer using the original data file as the shared reference input.

Good-enough for translation should be stricter than good-enough for cleanup because translation can silently change behavior even when code compiles.

21.9 Tool Availability Check Before High-Risk Modes

Before running high-risk or tool-dependent modes such as translate_cpp, the pipeline should check available tools and pass that information to the Planner and Test Designer.

- Check Python: python --version.
- Check test tools: python -m pytest --version when pytest is expected.
- Check CMake: cmake --version or where cmake on Windows.
- Check compiler: where cl, where g++, or equivalent.
- Check shell capabilities: Windows cmd, PowerShell, bash, or WSL.
- If a required tool is unavailable, the workflow may still create a candidate, but the final status should honestly say validation is blocked by environment limitations.

21.10 Model Upgrades Help, But Workflow Design Is Foundational

Better models reduce errors, but they do not remove the need for stage boundaries, schemas, validation, rollback, and environment-aware tooling. The tested system improved not only because the reviewer model was upgraded, but because the workflow became more precise about what each stage was allowed to judge.

- Use stronger models for high-judgment roles such as Planner and Reviewer when possible.
- Do not rely on model strength to replace deterministic validation.
- Do not rely on a better model to fix ambiguous prompts or missing stage boundaries.
- Preserve architecture/model routing so stronger models can be swapped in later without redesigning the workflow.

21.11 Agent Skills and Prompt Packs Must Be Evaluated, Not Assumed Helpful

Recent benchmark work on agent skills suggests that injected procedural guidance can help in some specialized cases, but many generic skills add token overhead without improving pass rate. For this project, that means guidelines should be useful, concise enough to be actionable, and compatible with the current repo context.

- Use mode-specific guidance rather than dumping every possible rule into every prompt.
- Keep the long guideline as the source of truth, but pass concise mode summaries to individual agents when context is limited.
- Evaluate whether a new guideline section actually improves outcomes on representative test runs.
- Avoid outdated or environment-mismatched instructions that conflict with the current repo or toolchain.

23. Updated Final Operating Model

The refined operating model after testing and literature review is:

1. Planner defines the intended outcome and what good enough means for this pass.

2. Plan Reviewer judges whether the plan is coherent enough to attempt, without requiring apply or validation results.

3. Test Designer defines the evidence needed to prove the good-enough claim.

4. Applier performs only enough work to satisfy the approved plan and workflow mode.

5. Validator executes deterministic checks and records objective evidence.

6. Repair Agent fixes validation failures without broadening scope.

7. Final Reviewer judges whether the outcome, explanation, and evidence match.

8. If validation or review fails, the candidate is reverted and the report explains why.

The system should be summarized as evidence-based autonomy: the LLM is allowed to decide scope, approach, and sufficiency, but every autonomous decision must be tied to repo context, guidelines, validation commands, test output, and reviewer judgment.

24. Additional Research References Added

These references informed the refinement sections above. They are included as research inspiration, not as rigid requirements.

- Yang et al. (2024), “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.” NeurIPS 2024. Key idea: the agent-computer interface materially affects repository navigation, editing, and test execution performance.
- Jimenez et al. / SWE-bench and SWE-bench Verified. Key idea: realistic software engineering evaluation requires real repositories, issue-like tasks, patch validation, and test-based evidence rather than isolated snippets.
- Recent SWE-bench extensions such as SWE-Bench Pro and SWE-EVO. Key idea: long-horizon, repo-level work requires planning, multi-file coordination, environment realism, and robust validation.
- Roychoudhury, “Agentic AI Software Engineers: Programming with Trust.” Key idea: trust, oversight, validation, and guardrails are central to agentic software engineering.
- SWE-Skills-Bench (2026). Key idea: injected skills/guidelines are not automatically helpful; their utility depends on domain fit, abstraction level, and compatibility with the current task context.
- Empirical studies of agent-authored pull requests and code review dynamics. Key idea: reviewers often focus on correctness, overengineering, maintainability, and evidence, which supports strong final review and traceability.